

Fakultet tehničkih nauka, Novi Sad

Inženjerstvo informacionih sistema

Eksploatacija, održavanje i nadogradnja informacionih sistema (EONIS)

PROJEKTNA DOKUMENTACIJA

Veb prodavnica parfema

SCENTIQ

Student: Marija Nenadić, IT49/2021

Izvorni kod: github.com/MarijaNenadic02/EONIS-Projekat

Novi Sad, 2026.

Projektna dokumentacija

Predmet: Eksploatacija, održavanje i nadogradnja informacionih sistema 2025/26. **Tema:** Veb prodavnica **Naziv sistema:** SCENTIQ, prodavnica parfema **Repozitorijum (izvorni kod):** github.com/MarijaNenadic02/EONIS-Projekat

Sadržaj

1. [Opis realnog sistema](#)
 2. [Korišćene tehnologije](#)
 3. [UML dijagrami](#)
 4. [Baza podataka](#)
 5. [Opis predloženog rešenja](#)
 6. [Zaključak](#)
-

1. Opis realnog sistema

SCENTIQ je veb prodavnica namenjena prodaji parfema. Sistem oslikava rad jedne manje parfimerije koja svoju ponudu želi da prebaci na internet, tako da kupci mogu da pregledaju proizvode, dodaju ih u korpu i plate karticom, dok zaposleni preko administracije održavaju ponudu i prate prodaju.

U realnom poslovanju razlikujemo dve grupe učesnika. Prva grupa su kupci, koji dolaze na sajt, pretražuju parfeme po brendu, vrsti i ceni, čitaju opise i na kraju kupuju ono što im odgovara. Druga grupa su zaposleni, odnosno administrator, koji unose nove proizvode, menjaju cene i stanje na lageru, prate pristigle porudžbine i proveravaju koje su uplate realizovane.

Ključni poslovni procesi koje sistem pokriva su:

- pregled i pretraga ponude parfema,
- vođenje korpe za svakog prijavljenog korisnika,
- naručivanje i plaćanje karticom,
- praćenje stanja na lageru i njegovo automatsko umanjivanje nakon prodaje,
- administracija proizvoda, brendova, kategorija, korisnika i porudžbina,
- evidencija uplata i uvid u realizovane transakcije.

Sistem vodi računa o poslovnim pravilima koja postoje i u stvarnoj prodavnici. Najvažnije je da se ne može poručiti više komada nego što ih ima na stanju, kao i da se stanje umanjuje tek kada je uplata zaista izvršena. Na taj način podaci o zalihama ostaju tačni.

2. Korišćene tehnologije

Aplikacija je razvijena kao celina sa dva dela: serverski deo (backend) koji nudi REST servise i klijentski deo (frontend) koji predstavlja korisnički interfejs. Oba dela su pisana u JavaScript okruženju, pa se kroz ceo projekat koristi isti jezik.

Serverski deo:

- Node.js i Express za izradu REST servisa,
- Prisma ORM uz SQLite bazu podataka, po code-first pristupu,
- JSON Web Token (JWT) za prijavu i autorizaciju,
- bcrypt za čuvanje lozinki u vidu heša,
- Zod za proveru ispravnosti ulaznih podataka,
- Multer za otpremanje slika proizvoda,
- Stripe za obradu plaćanja karticom, u testnom režimu.

Klijentski deo:

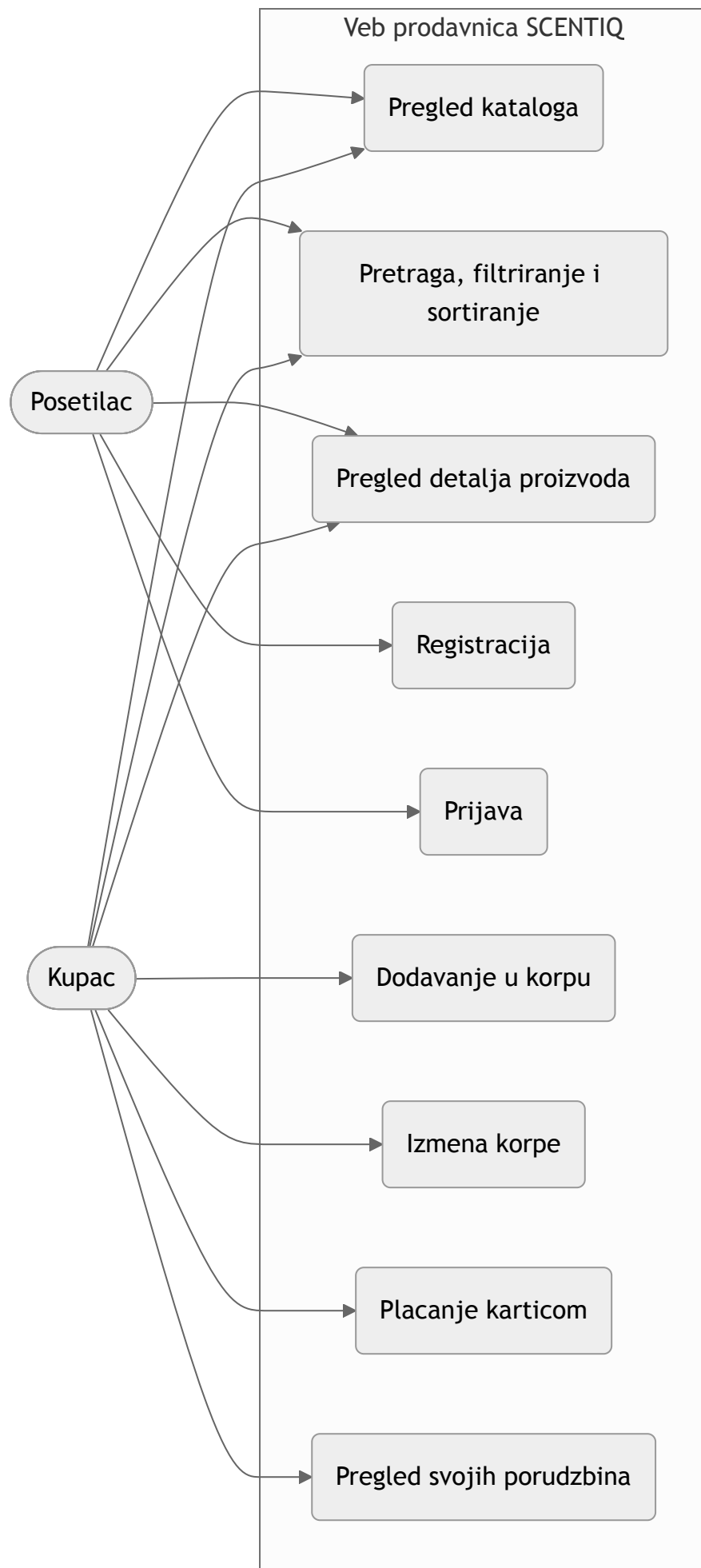
- React uz alat Vite,
- React Router za navigaciju između stranica,
- TanStack Query za rad sa podacima sa servera, paginaciju i keširanje,
- React Hook Form i Zod za forme i validaciju,
- Tailwind CSS za izgled.

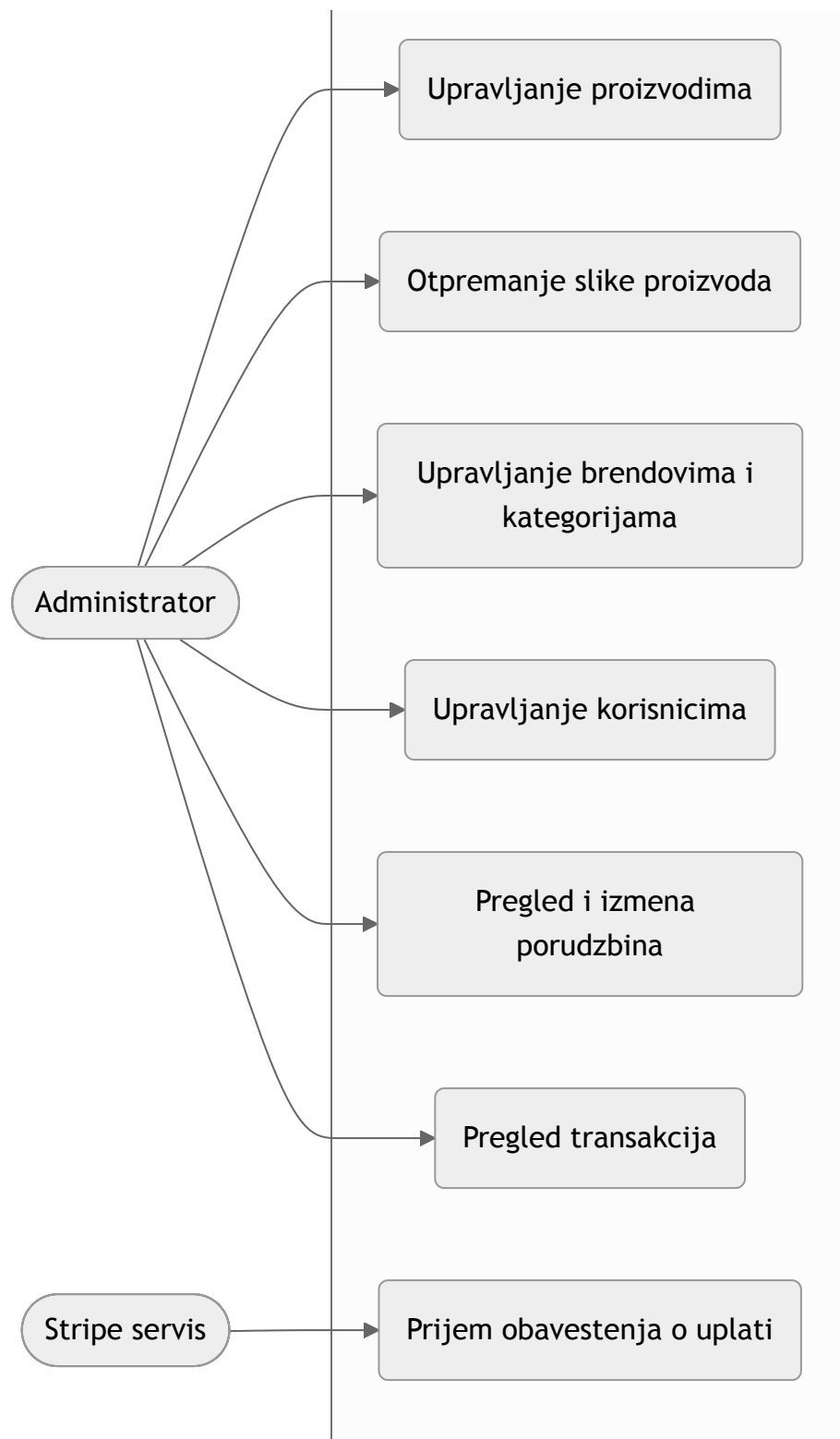
Izbor SQLite baze uz code-first pristup je napravljen zbog jednostavnosti pokretanja. Baza je jedan fajl, ne zahteva poseban server, pa se ceo sistem lako pokreće na bilo kom računaru, što je pogodno za demonstraciju i odbranu.

3. UML dijagrami

3.1. Dijagram slučajeva upotrebe

Dijagram prikazuje glavne aktere i radnje koje oni mogu da izvrše. Posetilac je neprijavljeni korisnik, Kupac je prijavljeni korisnik sa ulogom CUSTOMER, a Administrator je korisnik sa ulogom ADMIN. Stripe je spoljni sistem koji sistemu šalje obaveštenje o uspešnoj uplati.





Administrator nasleđuje sve mogućnosti kupca, jer je i on prijavljeni korisnik, a uz to ima i pristup administraciji.

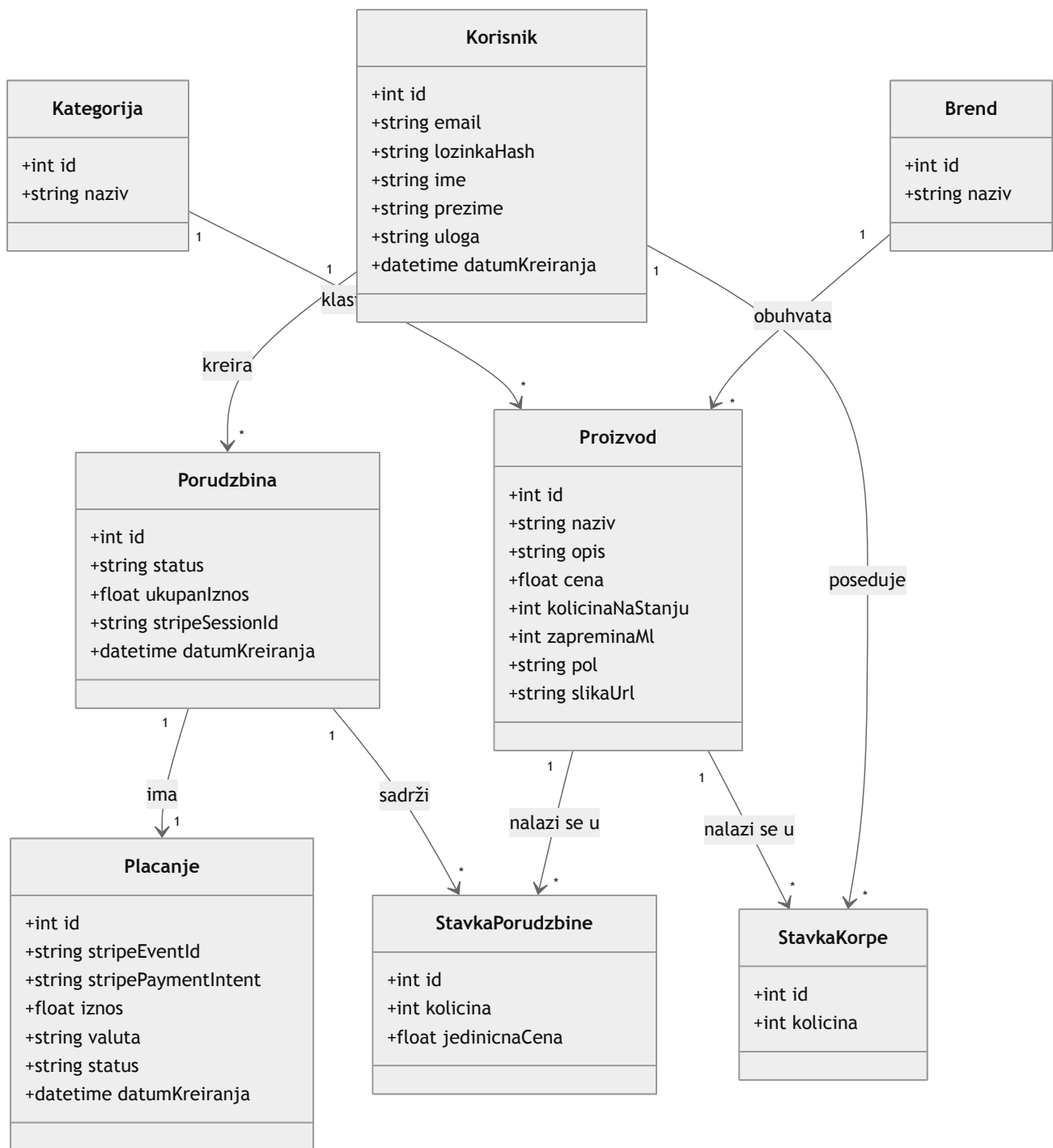
Opisi važnijih slučajeva upotrebe:

Slučaj upotrebe	Akter	Opis
Pregled kataloga	Posetilac, Kupac	Prikaz liste parfema sa slikom, brendom, cenom i stanjem.

Slučaj upotrebe	Akter	Opis
Pretraga, filtriranje i sortiranje	Posetilac, Kupac	Pronalaženje proizvoda po nazivu i brendu, filtriranje po brendu, vrsti, polu i ceni, kao i sortiranje po ceni i nazivu.
Registracija	Posetilac	Otvaranje novog naloga sa ulogom kupca, uz proveru ispravnosti podataka.
Prijava	Posetilac	Prijava postojećeg korisnika i izdavanje tokena za pristup.
Dodavanje u korpu	Kupac	Dodavanje proizvoda u korpu, uz proveru da tražena količina ne prelazi stanje na lageru.
Izmena korpe	Kupac	Promena količine ili izbacivanje proizvoda iz korpe.
Plaćanje karticom	Kupac	Pokretanje plaćanja preko Stripe naplatne stranice i kreiranje porudžbine.
Pregled svojih porudžbina	Kupac	Uvid u sopstvene porudžbine i njihov status.
Upravljanje proizvodima	Administrator	Dodavanje, izmena i brisanje parfema.
Otpremanje slike proizvoda	Administrator	Postavljanje slike proizvoda sa računara.
Upravljanje brendovima i kategorijama	Administrator	Održavanje šifarnika brendova i vrsta parfema.
Upravljanje korisnicima	Administrator	Pregled, dodavanje, izmena i brisanje korisnika.
Pregled i izmena porudžbina	Administrator	Uvid u sve porudžbine i promena njihovog statusa.
Pregled transakcija	Administrator	Uvid u realizovane uplate sa podacima o kupcu, proizvodu, ceni i količini.
Prijem obaveštenja o uplati	Stripe	Slanje webhook obaveštenja sistemu nakon uspešne uplate.

3.2. Dijagram klasa

Dijagram klasa prikazuje statičku strukturu sistema. Osnovne klase su Korisnik, Proizvod, Brend, Kategorija i Porudzbina. Klase StavkaKorpe i StavkaPorudzbine su asocijativne klase, jer povezuju dve druge klase i nose dodatne podatke (količinu, a kod stavke porudžbine i jediničnu cenu). Klasa Placanje čuva podatke o uplati koja stiže preko Stripe servisa.



Opisi relacija između klasa:

- Jedan Korisnik može da ima više porudžbina, a svaka porudžbina pripada tačno jednom korisniku.
- Jedan Korisnik može da ima više stavki u korpi. Svaka stavka korpe povezuje jednog korisnika i jedan proizvod i čuva količinu, pa predstavlja asocijativnu klasu između korisnika i proizvoda.
- Jedan Brend obuhvata više proizvoda, a svaki proizvod pripada jednom brendu.
- Jedna Kategorija (vrsta parfema) obuhvata više proizvoda, a svaki proizvod pripada jednoj kategoriji.

- Jedna Porudzbina sadrži više stavki porudžbine. Stavka porudžbine povezuje jednu porudžbinu i jedan proizvod i čuva količinu i jediničnu cenu u trenutku kupovine, pa je takođe asocijativna klasa.
- Jedna Porudzbina ima tačno jedno Plaćanje, koje nastaje kada Stripe potvrdi uplatu.

Uloga korisnika (ADMIN ili CUSTOMER) čuva se kao polje u klasi Korisnik, pa nije potrebna posebna tabela za administratora. Time je ispunjen zahtev da sistem ima najmanje dve uloge.

4. Baza podataka

U projektu je izabran code-first pristup. To znači da se struktura baze prvo opisuje u kodu, kroz Prisma model, a zatim se iz tog modela generišu tabele u bazi. Poslovna logika se ne realizuje kroz okidače (trigere) u bazi, već kroz programski kod na serveru, što code-first pristup i podrazumeva.

Proces izrade baze tekao je ovako:

1. U fajlu `server/prisma/schema.prisma` opisani su modeli (Korisnik, Brend, Kategorija, Proizvod, StavkaKorpe, Porudzbina, StavkaPorudzbine, Plaćanje), njihova polja i veze između njih.
2. Komandom `prisma migrate` Prisma na osnovu modela kreira migraciju, odnosno SQL skriptu koja pravi tabele, i primenjuje je na SQLite bazu. Migracije se čuvaju u projektu, pa se baza na drugom računaru dobija istom komandom.
3. Komandom `prisma generate` generiše se Prisma klijent, kroz koji kod na serveru čita i upisuje podatke na siguran način.
4. Skripta `server/prisma/seed.js` puni bazu početnim podacima: naložima administratora i kupca, listom brendova i kategorija i nekoliko parfema.

Pošto SQLite ne podržava nabrojive tipove (enum), polja kao što su uloga korisnika, pol proizvoda i status porudžbine čuvaju se kao tekst, a dozvoljene vrednosti se proveravaju u kodu, kroz Zod validaciju.

Poslovna logika u kodu:

- Pri dodavanju u korpu i pri naručivanju proverava se da tražena količina ne prelazi stanje na lageru. Ako prelazi, server vraća grešku i radnja se odbija.
- Stanje na lageru se umanjuje tek kada Stripe potvrdi uplatu, u okviru obrade webhook obaveštenja. U istoj operaciji porudžbina dobija status PAID, beleži se uplata i prazni se korpa kupca. Sve to se izvršava kao jedna transakcija, da podaci ne bi ostali u polovičnom stanju.

5. Opis predloženog rešenja

Arhitektura

Rešenje je podeljeno na dva dela. Serverski deo je REST aplikacija koja izlaže servise pod putanjom `/api`, a klijentski deo je jednostranična React aplikacija koja te servise poziva. Takva podela razdvaja logiku od prikaza i omogućava da se delovi razvijaju i testiraju nezavisno.

Serverski deo

Server je organizovan po celinama, gde svaka celina pokriva jedan deo sistema (autentifikacija, proizvodi, brendovi i kategorije, korpa, porudžbine, korisnici, plaćanja, otpremanje slika). Za svaku tabelu su realizovane CRUD operacije, odnosno kreiranje, čitanje, izmena i brisanje.

Nad listama proizvoda, korisnika i porudžbina podržani su pretraga, filtriranje, sortiranje i paginacija, tako da se i veće količine podataka pregledaju pregledno.

Obrada izuzetaka je centralizovana. Postoji jedan sloj koji hvata sve greške, bilo da su nastale pri validaciji, da traženi zapis ne postoji ili da je prekršeno neko poslovno pravilo, i vraća jasan odgovor sa odgovarajućim HTTP statusnim kodom. Time se izbegava da neobrađena greška sruši aplikaciju.

Autentifikacija i autorizacija

Prijava radi preko JSON Web Tokena. Nakon uspešne prijave korisnik dobija token, koji se uz svaki naredni zahtev šalje serveru. Server iz tokena čita ko je korisnik i koju ulogu ima. Lozinke se nikada ne čuvaju u izvornom obliku, već kao heš dobijen pomoću bcrypt funkcije.

Autorizacija se zasniva na ulogama. Rute za administraciju dostupne su samo korisniku sa ulogom ADMIN, dok su rute za korpu i porudžbine dostupne prijavljenom kupcu. Pokušaj pristupa bez odgovarajuće uloge vraća grešku zabranjenog pristupa.

Klijentski deo

Korisnički interfejs je podeljen na javni deo, deo za kupca i administraciju. Javni deo čine katalog i stranica proizvoda. Deo za kupca obuhvata korpu, plaćanje i pregled porudžbina. Administracija je zaštićena i sadrži upravljanje proizvodima, brendovima, kategorijama, korisnicima i porudžbinama, kao i pregled transakcija.

Sve forme imaju validaciju, pa korisnik dobija jasnu poruku kada unese neispravan podatak. Podaci sa servera se učitavaju i keširaju kroz TanStack Query, što olakšava paginaciju i osvežavanje prikaza.

Plaćanje preko Stripe servisa

Kada kupac krene u plaćanje, server na osnovu sadržaja korpe kreira Stripe sesiju za plaćanje i kreira porudžbinu u statusu PENDING. Kupac se preusmerava na Stripe naplatnu stranicu, gde unosi podatke o kartici. U testnom režimu koristi se testna kartica.

Nakon uspešne uplate Stripe šalje webhook obaveštenje serveru. Server proverava ispravnost tog obaveštenja, pa porudžbinu prebacuje u status PAID, umanjuje stanje na lageru, beleži uplatu i prazni korpu. Administrator u svom panelu, u delu sa transakcijama, vidi koji je proizvod kupljen, po kojoj ceni, u kojoj količini i koji ga je kupac kupio.

Otpremanje slika

Administrator sliku proizvoda može da postavi otpremanjem fajla sa računara. Server prima fajl, čuva ga i vraća putanju koja se zapisuje uz proizvod, pa se slika prikazuje u prodavnici.

6. Zaključak

Razvijena je veb prodavnica parfema koja pokriva sve zahteve projektnog zadatka. Sistem ima jasno razdvojene uloge kupca i administratora, potpune CRUD operacije nad svim tabelama, pretragu, filtriranje, sortiranje i paginaciju, prijavu sa autentifikacijom i autorizacijom, kao i obradu izuzetaka.

Poslovna logika je rešena u kodu, kroz code-first pristup. Provera stanja na lageru i umanjivanje zaliha nakon uplate obezbeđuju da podaci o zalihama ostanu tačni. Plaćanje je realizovano preko Stripe servisa u testnom režimu, sa obradom webhook obaveštenja i prikazom transakcija administratoru.

Rešenje je napravljeno tako da se lako pokreće i nadograđuje. Pošto su delovi sistema razdvojeni, moguće je dodati nove mogućnosti, na primer ocene proizvoda, popuste ili izveštaje o prodaji, bez velikih izmena na postojećem kodu. Time sistem predstavlja dobru osnovu za pravu prodavnicu parfema.